CrewCheck project handbook

MVP implementation and review guide

Version 2.0 | 10 November 2026 | Stakeholder review

CrewCheck reviews crew rosters against explicit rules, shows the evidence behind each finding and keeps every result with the rule version that produced it. This handbook explains the current application from first use through technical handover.

The application is ready for fictional-data review. Operational use depends on an approved rulebook, authoritative roster and qualification data, company sign-in and an agreed operating model.

Review topics

Requested topic	Handbook section
Proposed user workflow	4 User workflow
How a scheduler uploads a roster	5 Roster upload
How violations are displayed	6 Findings, evidence and timeline
How managers view historical flags and uploads	7 History and import log
How rules and thresholds are configured	8 Rule builder
Outstanding risks or unresolved questions	18 Decisions and handover

Reading guide

Sections 2–8 cover scope, architecture and user tasks. Sections 9–12 specify data, rules, storage and API behaviour. Sections 13–17 cover installation, Docker and databases, verification, operations and the internal deployment plan.

What changed in version 2.0

The separately hosted demo was retired; the FastAPI application is now the single implementation. New since 1.4: a crew timeline, display time zones, a manager rule builder with impact preview, an import log that keeps rejected files and original uploads, automatic daylight-saving handling, a database layer that runs on SQLite or PostgreSQL, and Docker packaging.

2 Scope and access

CrewCheck accepts CSV and Excel rosters, validates them, shows a preview, runs deterministic checks and keeps per-duty evidence, the result history and the rule versions. A person reviews the findings and decides what to do next.

Capability	Scheduler	Manager
Upload and check CSV or XLSX	Yes	Yes
Read results, table and timeline	Own checks	All checks
Import log and original files	Own attempts	All attempts
Read the active rule set	Yes	Yes
Build rules, preview impact, save versions	No	Yes
Edit a roster or resolve a flag in the app	No	No

Ways to run it

Setup	Application and data	Sign-in
Local	FastAPI with a SQLite file	Two seeded demo accounts
Docker	Same image; SQLite or PostgreSQL; data in volumes	Two seeded demo accounts
Planned internal server	Same application on PostgreSQL	Company authentication and assigned roles

Excluded from this MVP

PDF, DOCX, legacy .xls and macro-enabled workbooks are outside input scope. The app does not edit schedules, notify crew, assign remediation, approve exceptions or use an LLM. It is not an implementation of a specific aviation regulation, does not merge duty history across uploads and does not hold a full qualification registry.

3 System architecture

One FastAPI process serves the browser interface, the JSON API and the project documentation. The browser and API share an origin; no other services are required.

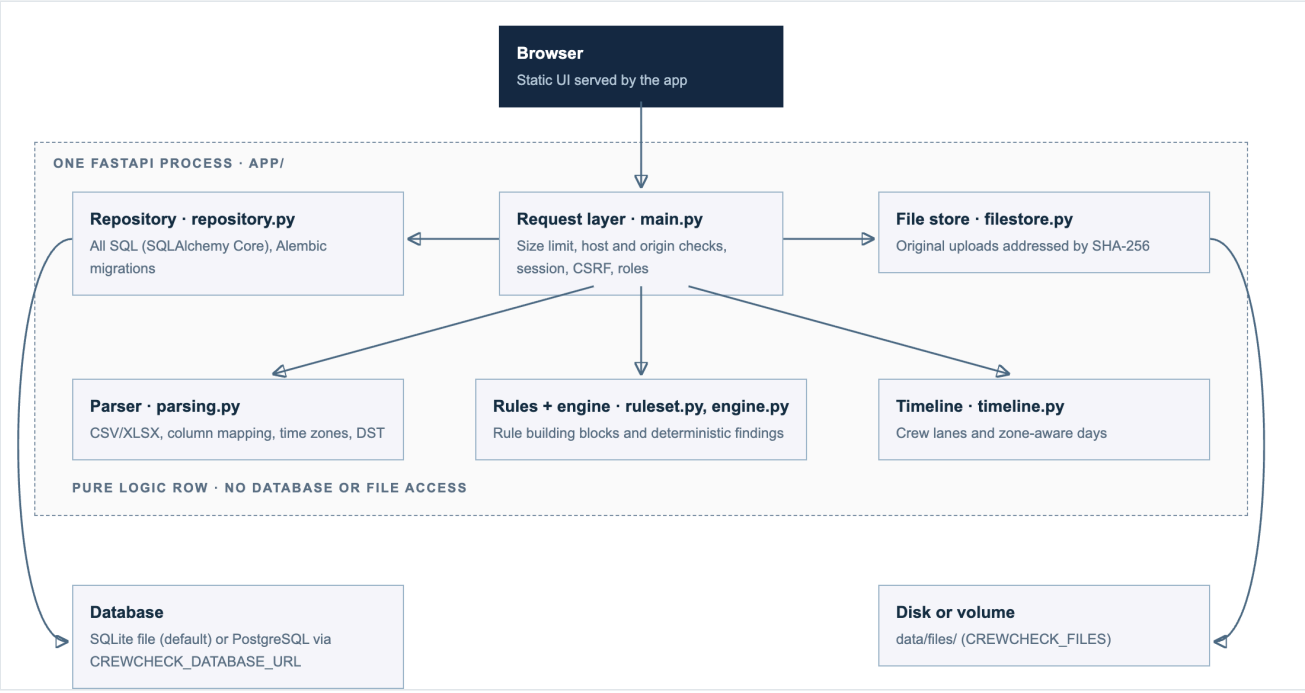


Figure 1 Components inside the application process and its storage

The request layer enforces request size, trusted host names, same-origin writes, sessions, CSRF tokens and roles. The parser, rule engine and timeline are pure functions: they never read or write storage, which keeps findings reproducible and testable. All SQL lives in the repository module, and original files go through the file store.

Request lifecycle

1. Upload a file. The server stores its bytes by SHA-256 and records the attempt in the import log, whether or not the file is valid.
2. The parser validates every row and normalizes times to UTC. If it needs a worksheet, column mapping or source time zone it asks for them; invalid rows reject the whole file.
3. An accepted file is saved as normalized rows together with the active rule version, then shown as a preview.
4. Run the check. The engine applies that rule version and the result is saved once; repeating the request returns the same result.
5. Later views read the saved result. New rule versions apply only to new uploads.

4 User workflow

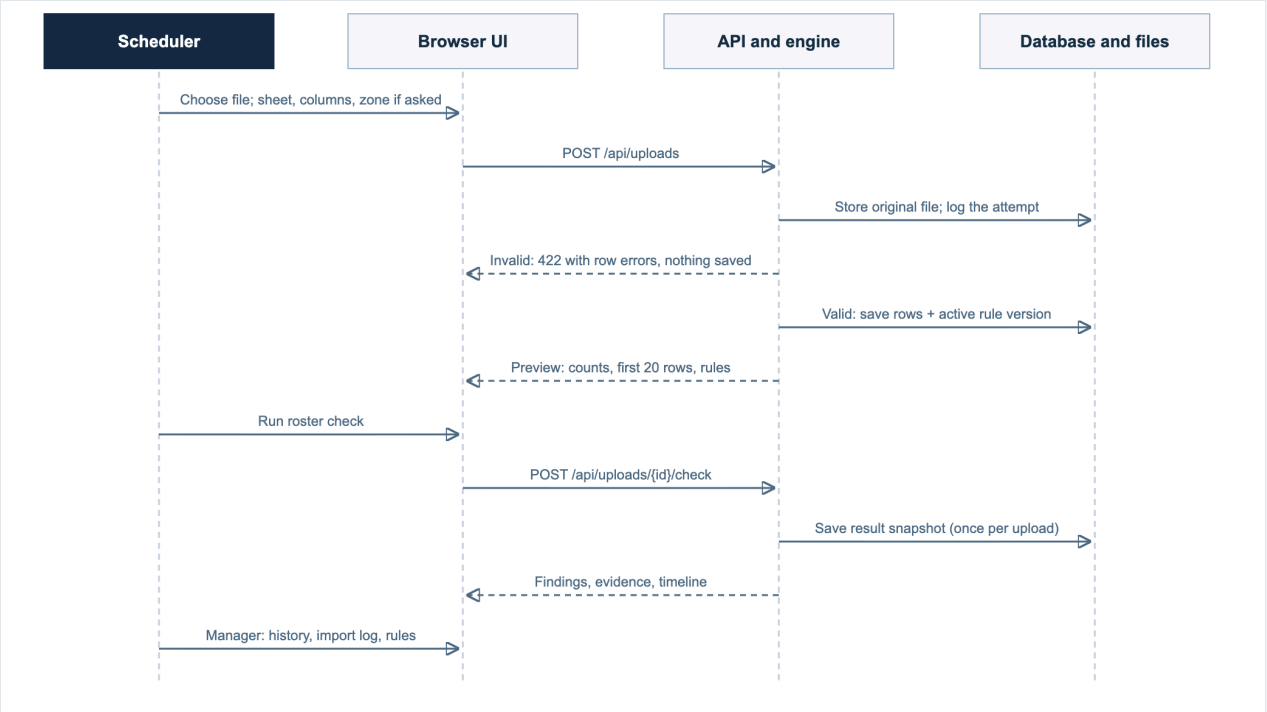


Figure 2 Upload and check sequence (solid: request, dashed: response)

Scheduler journey

Sign in as Scheduler and open Load roster. Choose a file; answer the worksheet, column or time-zone questions if asked; review the preview and run the check. Filter findings, switch between Table and Timeline, and open Details for evidence. Correct the roster in the source spreadsheet and upload it again to compare.

Manager journey

Sign in as Manager to see every scheduler's checks and upload attempts. Reopen any saved check with its original evidence and rules, download an original file from the import log, and build or adjust rules. Preview a draft against a saved check, then save it as a new version with a reason.

States

Every upload attempt is logged as accepted, rejected or needs setup. Only an accepted upload becomes a preview; a successful check creates exactly one result. A preview can remain unchecked if the user leaves. Findings have no acknowledgement or resolution state.

5 Roster upload

1. Open Load roster. Drop a CSV or Excel file on the upload area, choose Browse files, or use the sample roster.
2. If asked, choose the worksheet, match source columns to the required fields, or confirm the source time zone, then continue.
3. Review the file name, duty and crew counts, the first 20 normalized rows and the rule set that will apply.
4. Choose Run roster check. The result is saved and displayed.

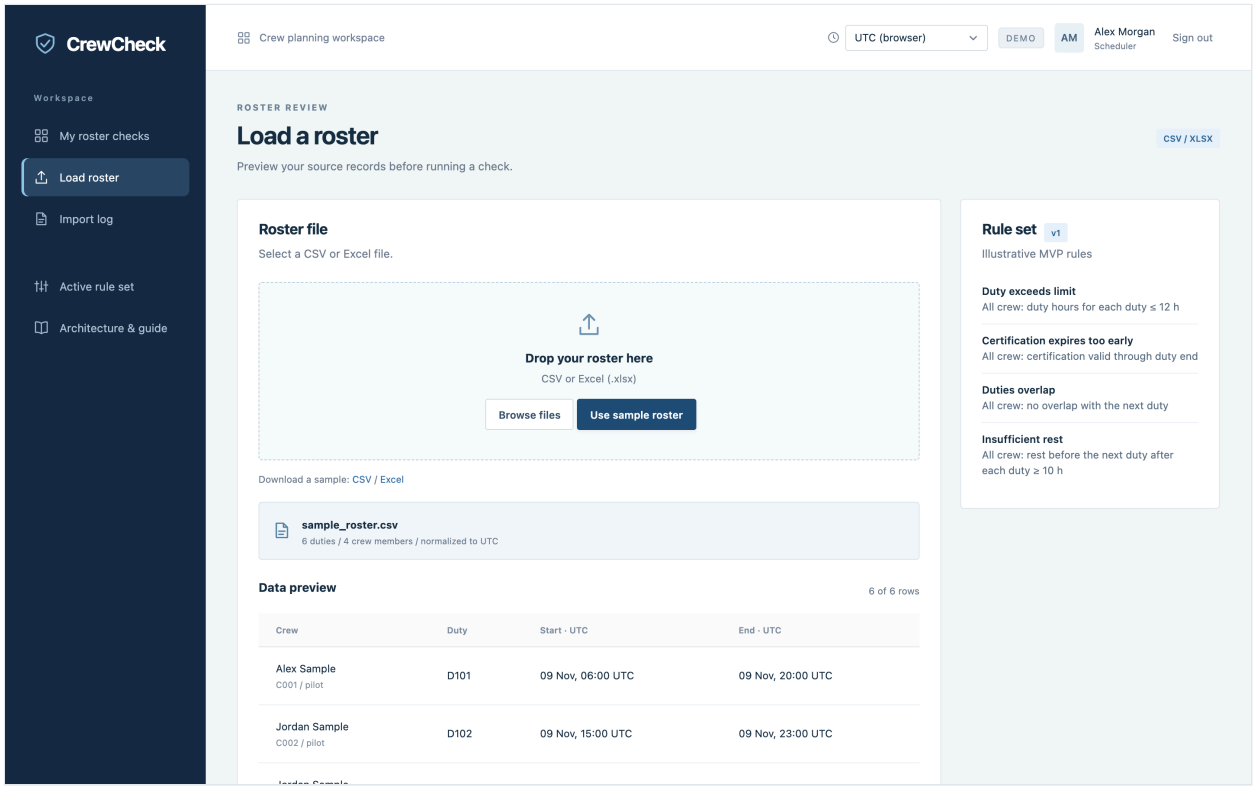


Figure 3 Upload preview with the rule set that will be applied

If validation fails, the message lists each row and problem. Correct the source file and upload again; nothing is checked from an invalid file. The attempt and its file stay in the import log.

Daylight-saving times

Local times in a named zone use the correct offset for their date. A time inside a clock change (repeated in autumn, skipped in spring) is resolved to the stricter reading: earliest duty start, latest duty end and earliest certification expiry. The preview warns about adjusted rows and each row's details explain the adjustment. An explicit offset in the cell always wins.

6 Findings, evidence and timeline

The result page shows duties checked, flagged duties, findings and crew count. A duty can have several findings, so flagged duties and findings are counted separately. Search by crew or duty ID, filter by role or show only flagged duties; the filters apply to both the table and the timeline.

Sample duty (sample_roster.csv)	Evidence	Finding with initial rules
D101	14 hours of duty	Above the 12-hour maximum
D103	7 hours between D102 and D103	Below the 10-hour minimum rest
D104	Expiry 12:00 UTC; duty ends 16:00 UTC	Certification expires before duty end

Reading the evidence

Details shows the observed and expected values, the times and duties involved, the rule code and version, the check ID, engine version and source file hash. For D103, D102 ends at 23:00 UTC on 9 November and D103 starts at 06:00 UTC on 10 November: 7 hours against a 10-hour minimum.

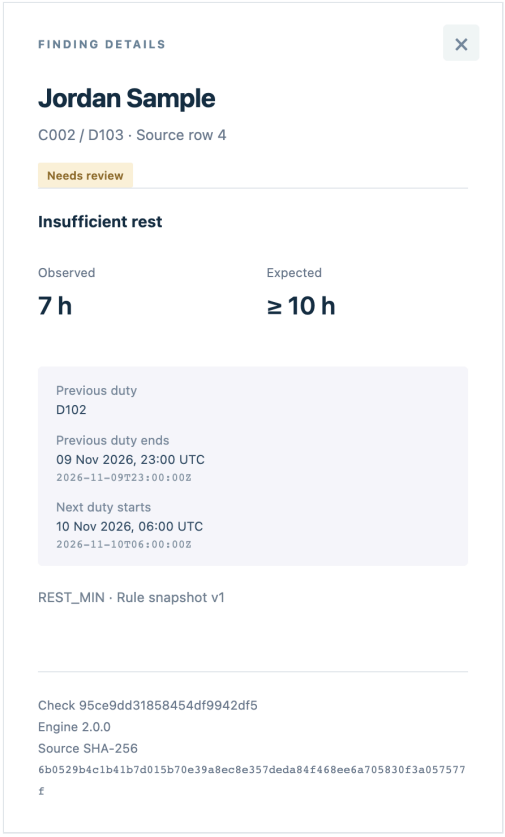


Figure 4 Evidence for D103

Times appear in the display time zone chosen in the top bar (the browser's zone by default) with the stored UTC value underneath. Missing certification is incomplete evidence, not proof of expiry; no flags means only that the configured rules found nothing.

6 Findings, evidence and timeline (continued)

Timeline

Timeline shows one row per crew member on a calendar. Bars are coloured by status (no flags, needs review, incomplete evidence); short rest and overlaps are hatched; each day header shows its finding count, and crew with the most findings come first. It fits the page by default and can be zoomed. Day columns follow the display time zone, so a daylight-saving day is 23 or 25 hours long. Select a bar to open its evidence.

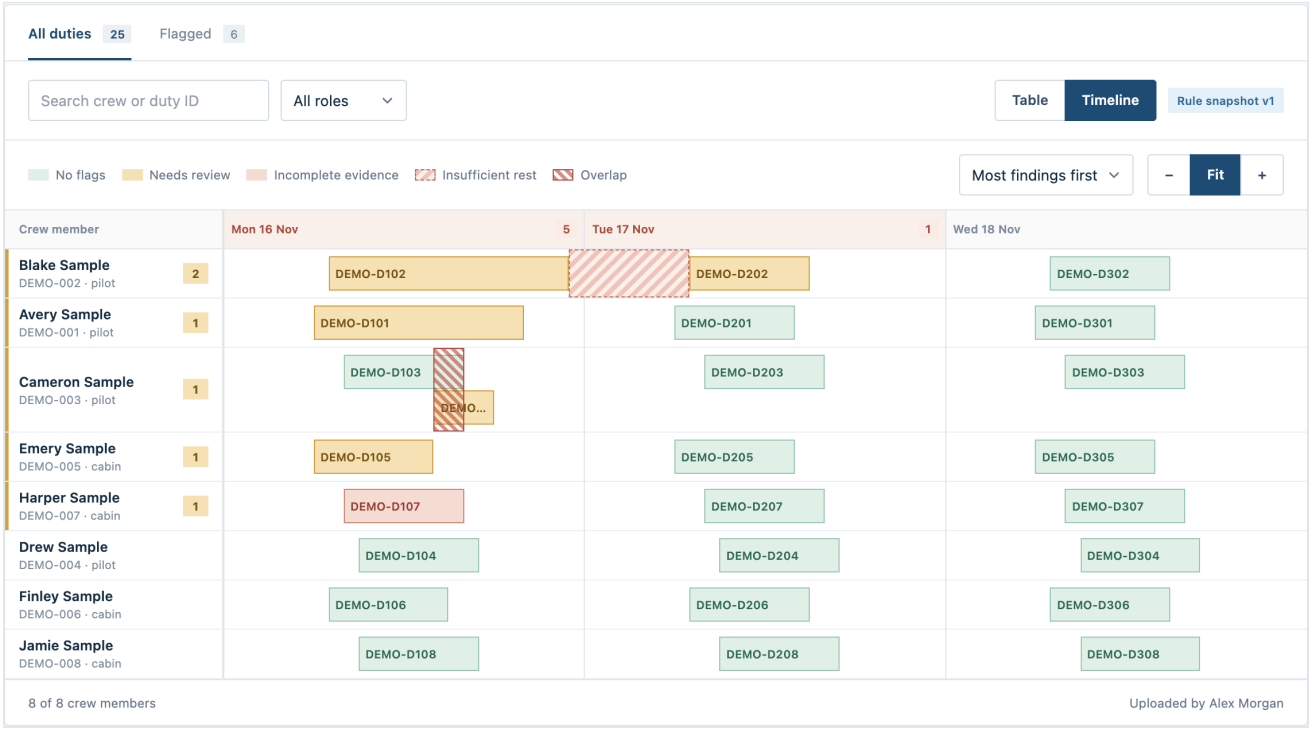


Figure 5 Timeline for review_roster.csv

In this example Blake Sample has a long duty followed by short rest (hatched), and Cameron Sample has two overlapping duties, drawn on separate rows with the overlap marked.

7 History and import log

Check history

Check history lists saved checks, 25 per page, newest first, with file, time, uploader, duty and finding counts and the rule version used. Search by file name or filter by the UTC date the check ran (not the duty dates). View check reopens the saved result exactly as it was produced; results are never recalculated.

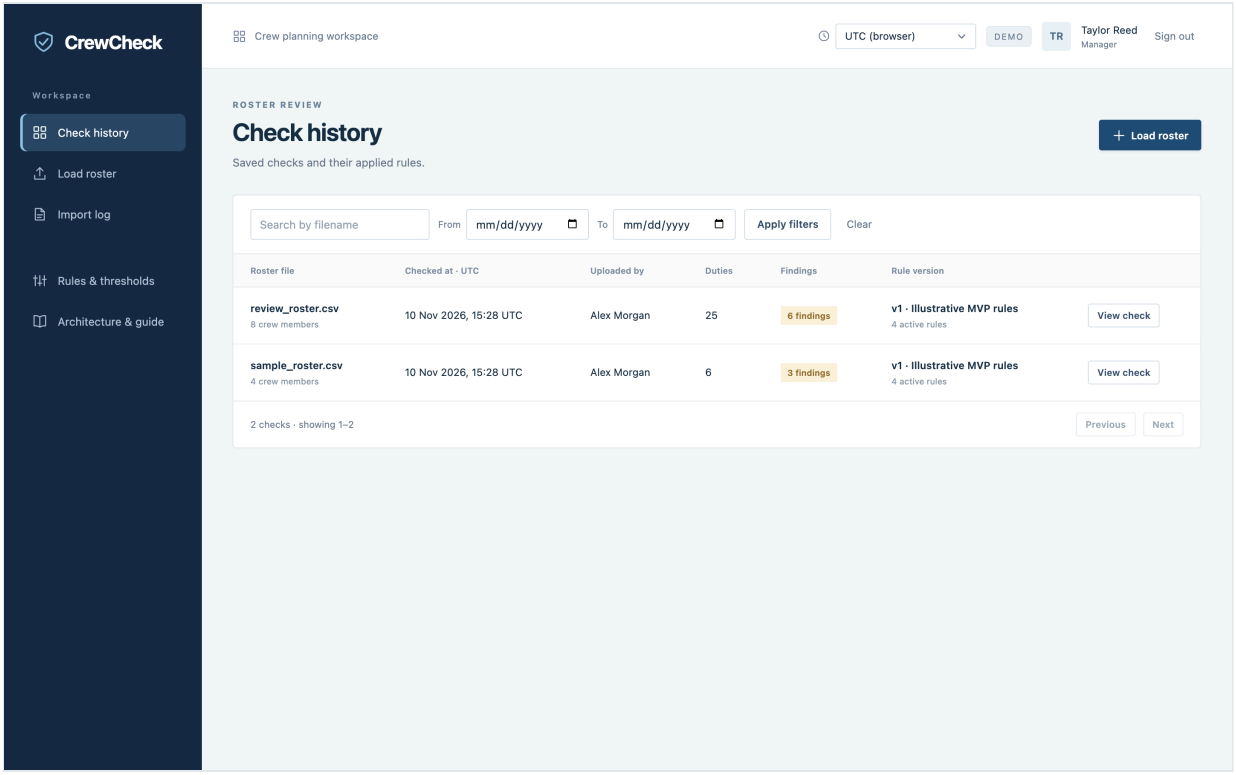


Figure 6 Manager view of check history

Repeating Run on one upload returns its existing result. Uploading the same file again creates a new attempt and can produce a separate comparison check.

7 History and import log (continued)

Import log

Import log lists every upload attempt with its status: Accepted (with duty count and a link to the check), Rejected (expand to see each row error) or Needs setup (which choice was missing). Original file downloads the exact bytes that were uploaded; identical files are stored once. Schedulers see their own attempts; managers see all.

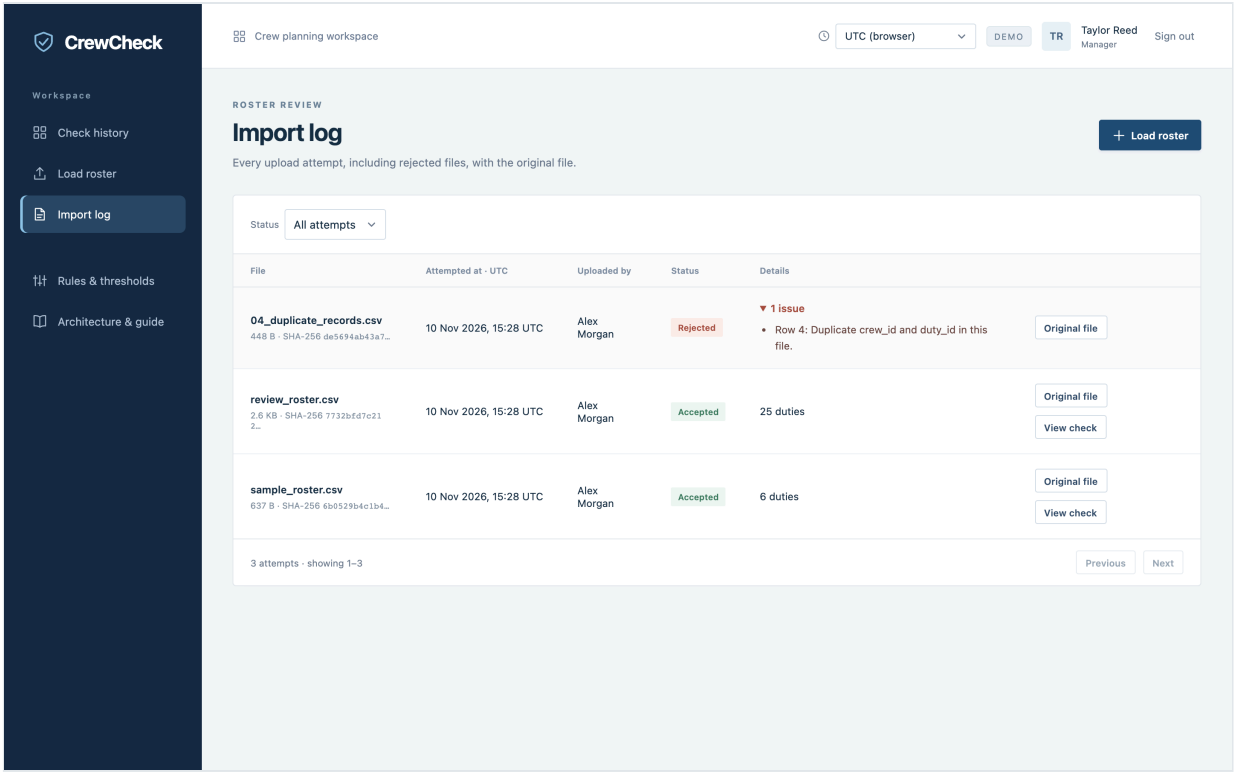


Figure 7 Import log with a rejected file and its error

8 Rule builder

Managers build rules without code. Each rule is a sentence assembled from fixed, tested parts, and the sentence is shown under the rule as it is edited. Schedulers can read the active rule set but cannot change it.

Part	Choices
Who	All crew, pilots or cabin crew
Only duties that…	Start, end or touch a daily time window; start on chosen weekdays; belong to listed crew IDs (up to five conditions, all must match)
Measure	Duty hours, number of duties, rest before the next duty, overlap with the next duty, consecutive duty days, certification validity left
Checked	For each duty, on any calendar day, in any rolling window of N days, after each duty, or in a row
Must be	At most, less than, at least or more than a limit
Local time zone	Zone used for time windows, weekdays and calendar days

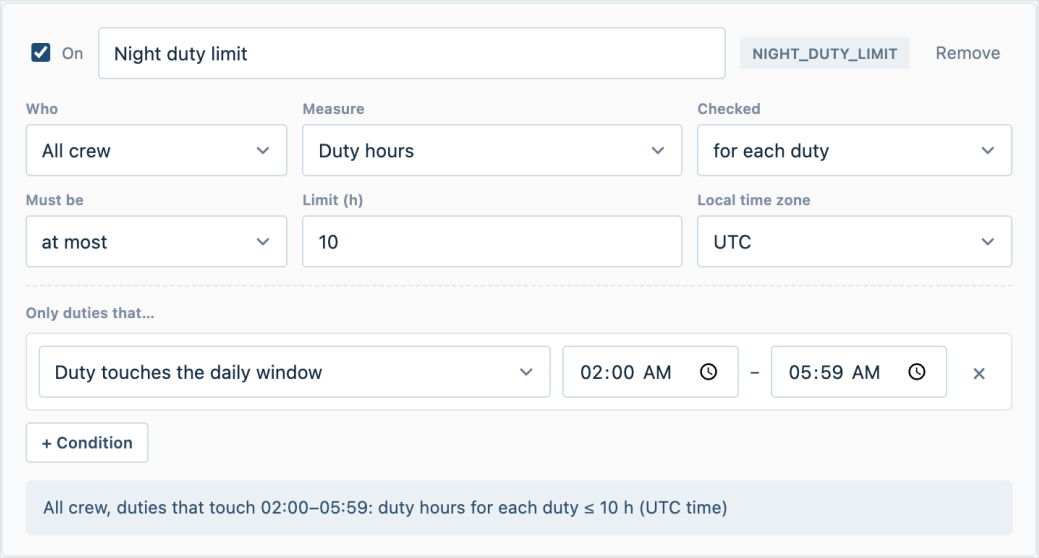


Figure 8 A night-duty rule built from the parts above

Templates

Add rule offers templates for the four built-in rules and for duty hours in any 7 or 28 days, duties on one calendar day, consecutive duty days, a night duty limit, rest after a night duty and a 30-day certification margin. Every value can be changed after adding.

8 Rule builder (continued)

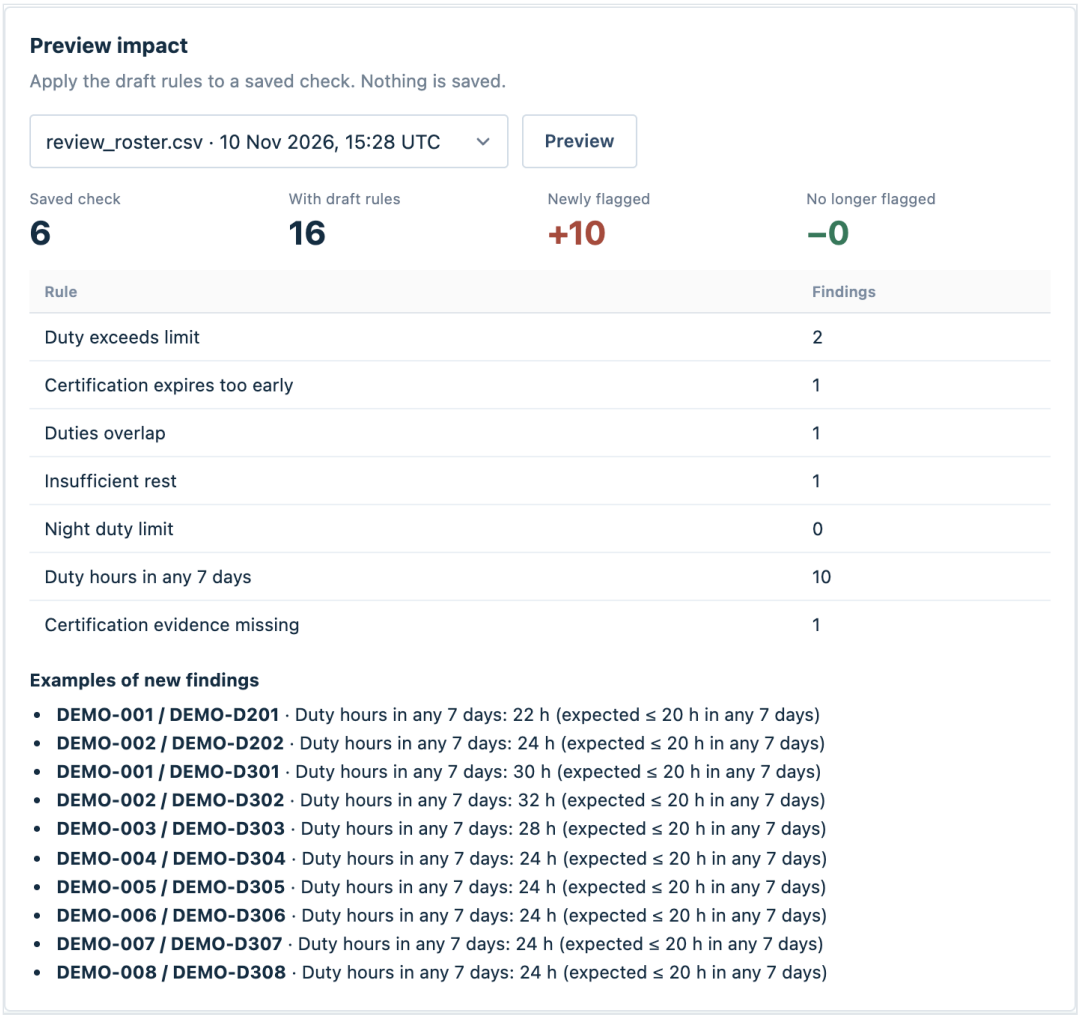


Figure 9 Impact preview of two draft rules against a saved check

Preview and save

Preview applies the draft rules to a saved check without saving anything and shows how many findings are added or removed, the count per rule and examples of new findings. Save new version requires a reason and records who saved it and when.

If another manager saved first, the save is refused with a conflict; reload and reapply the change. Existing checks keep the rules they were run with, and an open preview keeps the version it captured. Invalid input is reported per rule and field, for example a rolling window without a number of days.

Rules are stored as data and evaluated by the engine; nothing a manager enters is executed as code. A new kind of measure or condition is a development change with its own tests.

9 Input contract

Upload UTF-8 CSV or Excel .xlsx. Choose the worksheet in multi-sheet files and map differently named columns to the fields below; other columns are ignored. There is no fixed row limit and blank rows are skipped.

Column	Requirement and meaning
crew_id	Required. Stable identifier within the file, for example C001.
crew_name	Required. Consistent for a crew_id within the file.
role	Required. pilot or cabin (any letter case).
duty_id	Required. Unique together with crew_id within the file.
duty_start	Required date and time; Excel date cells are supported.
duty_end	Required, strictly after duty_start.
certification_expires_at	Optional. A missing value creates a missing-evidence finding.

Timestamps

Use Excel date/time cells or ISO-style text such as 2026-11-09 06:00 or 2026-11-09T06:00:00+09:00. Values without an offset need a source zone (for example UTC, Asia/Seoul or +09:00). Daylight-saving changes are handled as described in section 5. Times are stored in UTC.

Validation and capacity

Rows are rejected for missing required values, duplicate duties, inconsistent crew name or role, invalid periods, text over 160 characters or formulas in the selected sheet; up to 50 errors are reported. The upload size limit (default 5 MB, CREWCHECK_MAX_UPLOAD_BYTES) and workbook archive limits protect the server; they are not business rules.

10 Check semantics

The engine compares UTC instants using actual elapsed time, so durations across clock changes are correct. Duties are ordered per crew member by start and end, and rest is measured from the latest-ending earlier duty, which also handles nested overlaps. Decisions use unrounded values; displayed hours are rounded.

Built-in rule	Flag condition	Boundary
DUTY_LIMIT	Duty longer than the maximum (initially 12 h)	Equal passes
REST_MIN	Non-negative gap shorter than the minimum (initially 10 h)	Equal passes
DUTY_OVERLAP	Next duty starts before the latest earlier duty ends	Reported instead of rest
CERT_EXPIRED	Expiry before duty end (or start, if chosen)	Equal passes
CERT_MISSING	No expiry value	Incomplete evidence

How custom rules are measured

Measure	Meaning
Per calendar day	Hours are clipped at local midnight in the rule's zone; counts include duties touching that day.
Rolling N days	For each duty, the window ends at its end and covers the previous N days; partial duties are clipped.
After each duty	Conditions select the duty before the rest; the finding is placed on the next duty.
In a row	Consecutive local calendar days with any matching duty.

A duty receives at most one finding per rule. Rest before each crew member's first duty in a file is not assessed, and earlier uploads are not merged. No regulatory profile, report-time table, acclimatization model or exception approval is implemented.

Reproducibility

A saved result keeps the normalized rows, findings, summary, coverage note, engine version (2.0.0), source hash and rule version. Historical results are read, not recalculated. Rule versions saved in the version 1.x format are converted on read and give the same findings.

11 Storage and data model

Table	Contents and purpose
users, sessions, login_attempts	Demo accounts, 8-hour sessions with CSRF tokens, failed sign-in throttling.
rule_versions	Numbered rule sets with name, reason, author and time. Never updated.
uploads	Normalized rows, file name, source hash, owner and the captured rule version.
runs	Saved result for an upload; one per upload.
import_attempts	Every upload attempt: status, errors, settings, size, hash and link to the upload.
alembic_version	Applied database migration.

Database and files

SQLite is the default (data/crewcheck.db); PostgreSQL is used when CREWCHECK_DATABASE_URL is set. All SQL is in app/repository.py, tables are defined in app/schema.py and Alembic migrations in app/migrations run at startup. Databases created by earlier versions are adopted without changes. Times are ISO-8601 UTC text and snapshots JSON text, so saved evidence is identical on either database.

Original files are stored once per content hash under data/files/<first two characters>/<hash> (CREWCHECK_FILES). Files over the size limit are logged without their bytes.

Guarantees

Rule versions are numbered previous + 1 and protected by the primary key, so two simultaneous saves cannot both succeed. A unique upload reference ensures one result per upload. On SQLite, writes are serialized; on PostgreSQL, the same constraints apply. Results, rule versions and import records have no update or delete operation in the application, which is not protection against a database administrator. No retention or deletion schedule exists yet.

12 API reference

The browser uses same-origin JSON APIs (multipart for uploads). Writes require the X-CSRF-Token returned at sign-in, and cross-origin writes are refused.

Method and path	Contract
POST /api/auth/login, GET /api/auth/me, POST /api/auth/logout	Email and password; returns identity and csrf. Ten failed attempts in five minutes return 429.
POST /api/uploads	File plus optional options (worksheet, mapping, timezone). 201 with preview, rules, import_id and dst_adjusted_rows; 422 with errors or import_options.
POST /api/uploads/{id}/check	Run, or return the existing result.
GET /api/runs, GET /api/runs/{id}	History with q, from_date, to_date, offset (25 per page); a saved result.
GET /api/runs/{id}/timeline?tz=	Crew lanes and days in a time zone.
GET /api/imports, /api/imports/{id}, /api/imports/{id}/file	Import log with status and offset; one attempt; the original file.
GET /api/rules, /api/rules/history, /api/rules/catalog	Active rule set; versions (manager); rule building blocks and templates.
POST /api/rules/preview, POST /api/rules	Manager only. Dry run against a saved check; save a new version (409 on conflict).
GET /api/templates/{csv|xlsx}, GET /api/health	Sample roster download; status.

Errors

400: host name not allowed. 401: sign in again. 403: role, origin or CSRF failure. 404: missing or not accessible. 409: rule version conflict. 413: request too large. 422: file, settings or rule validation; the response explains which row or field. Full request and response shapes are in docs/api-contract.md; the generated schema is at /openapi.json.

13 Local setup

Python 3.11 or later and uv are required; the lockfile pins dependencies. Run from the project directory:

```
uv sync --frozen
./scripts/run.sh
```

Open http://127.0.0.1:8000 and sign in with a demo account. /documentation opens the project guide. Stop with Ctrl+C.

Role	Email	Initial demo password
Scheduler	scheduler@crewcheck.local	SchedulerDemo!2026
Manager	manager@crewcheck.local	ManagerDemo!2026

Configuration

Variable	Meaning
PORT	Default 8000.
CREWCHECK_DB	SQLite file. Default data/crewcheck.db.
CREWCHECK_DATABASE_URL	Use another database, e.g. postgresql+psycopg://user:pass@host:5432/crewcheck.
CREWCHECK_FILES	Folder for original files. Default: files next to the database.
CREWCHECK_ALLOWED_HOSTS	Host names the server answers to. Default localhost,127.0.0.1.
CREWCHECK_SCHEDULER_PASSWORD / _MANAGER_PASSWORD	Seed passwords, used only when accounts are first created.
CREWCHECK_SECURE_COOKIE	Set to 1 only behind HTTPS.
CREWCHECK_MAX_UPLOAD_BYTES	Upload size limit. Default 5 MB.

Export variables in the shell; .env.example lists them but is not loaded automatically. Use a new database path for an isolated demonstration. Account provisioning, password recovery and company SSO are not implemented.

14 Docker and databases

```
docker compose up -d --build                                     # SQLite
docker compose -f compose.yaml -f compose.postgres.yaml up -d --build   # PostgreSQL
docker compose logs -f
docker compose down          # stop; data is kept
docker compose down -v       # stop and delete data volumes
```

The image runs the application as a non-root user on port 8000 with a health check. The database and original files live in the crewcheck-data volume (/data); with the PostgreSQL override the database is in the crewcheck-postgres volume. Set CREWCHECK_ALLOWED_HOSTS to the server's host name and change the demo passwords before sharing access.

Switching databases

Set CREWCHECK_DATABASE_URL (and install the postgres extra outside Docker with uv sync --extra postgres). Migrations create or update tables at startup. Moving existing records between databases is a separate export and import step; it is not automated.

Schema changes

Change app/schema.py, then create a migration and review it before committing:

```
CREWCHECK_DATABASE_URL=sqlite:///data/crewcheck.db uv run alembic revision -m "describe
change"
```

15 Verification and acceptance

Verification on 10 November 2026: 209 tests passed on SQLite with 100% line and branch coverage, and the same suite passed on PostgreSQL 17. The application was exercised in Chrome and in Docker on both databases, including an existing database created by an earlier version.

Command	What it covers
uv run pytest --cov	Parsing, DST, rules and every rule building block, API access and roles, CSRF, import log, files, timeline, history, migrations and concurrency. Fails below 96% coverage.
CREWCHECK_TEST_DATABASE_URL=… uv run pytest	The whole suite against PostgreSQL, one fresh database per test.

Acceptance walkthrough

1. Sign in as Scheduler. Upload sample_roster.csv and confirm 6 duties and 4 crew. Run the check: 3 findings on 3 duties. Open D103 and confirm 7 hours of rest against 10.
2. Switch to Timeline and confirm the flagged duties and the hatched rest gap.
3. Upload samples/edge-cases/04_duplicate_records.csv. Confirm the row 4 error, then find the attempt and its original file in Import log.
4. Sign in as Manager. Reopen the scheduler's check from Check history.
5. In Rules & thresholds add Duty hours in any 7 days with a 20 h limit, preview it against the review roster check, and save with a reason. Upload again and confirm the new rule appears; the earlier check is unchanged.

Fixture	Expected with initial rules
sample_roster.csv / .xlsx	6 duties, 4 crew, 3 findings
review_roster.csv / .xlsx	25 duties, 8 crew, 6 findings
clean_roster.csv / .xlsx	24 duties, 8 crew, 0 findings
input_errors.csv / .xlsx	Asks for a time zone, then rejected for an invalid duration
edge-cases/06, 07	Exact boundaries pass; certification boundaries per edge-cases README

16 Operations and troubleshooting

Symptom	Action
Missing history	Check the signed-in role and the date filters; dates are when the check ran, in UTC.
File rejected	Open Import log for the row errors. Choose the sheet, map columns and give the source zone; replace formulas with values.
400 Invalid host	Add the host name to CREWCHECK_ALLOWED_HOSTS.
403 or session error	Refresh, sign in again and retry from the same address.
409 when saving rules	Reload, review the latest version and enter the change again.
Old check shows old findings	Expected. Upload again to apply the current rules.
Unexpected times	Check the display time zone in the top bar; stored values are UTC.

Backups

Back up the database and the files folder together. SQLite: use the online backup API or stop the app before copying; do not copy only the main file while it is running. PostgreSQL: use pg_dump. Docker: back up the crewcheck-data (and crewcheck-postgres) volumes. Backup automation, retention and restore tests are not yet in place.

Incidents

Record the time, route, non-sensitive error and version. Do not put real roster data or passwords in reports. /api/health reports that the app is running; use a sign-in and a fictional check to confirm storage.

17 Internal deployment plan

Already in place

The application runs on PostgreSQL through the repository layer and migrations, the test suite passes on PostgreSQL, Docker packaging exists and original files are retained in a replaceable file store.

Remaining sequence

1. Confirm the internal runtime, network boundary, company sign-in provider, teams, permissions and approved rule definitions.
2. Replace demo accounts with company authentication and record stable employee IDs as authors of uploads and rule versions.
3. Add a migration to timestamptz and jsonb if reporting needs them, keeping saved values identical.
4. Decide retention and deletion for records and original files; move files to managed object storage if required.
5. Automate backups and rehearse a restore. Load-test with representative roster sizes.
6. Validate an internal pilot with fictional or authorized test data, with agreed cutover and rollback conditions.

Acceptance criteria

Old checks remain identical, new rules affect only new uploads, users cannot read another team's records, repeated checks do not duplicate results, and record counts, hashes and rule references reconcile after any data transfer. Demo records should be discarded or kept only as a labelled test set.

18 Decisions and handover

Decision to confirm	Effect on the next phase
Which approved rulebook and version apply to each operation and crew role?	Defines the rules managers configure and any new measures needed.
What do duty start and end include, and are files complete rosters or revisions?	Defines calculations, prior-duty context and duplicate handling.
Which qualification records are authoritative?	Replaces the single expiry field.
Which sign-in provider, teams and user volumes apply?	Defines identity, authorization and sizing.
How long must records and original files be kept, and how are they deleted?	Defines storage lifecycle and backups.
Is acknowledgement, assignment, remediation or export of findings needed?	Defines whether findings become a managed workflow.

Handover package

This handbook (PDF and Word), the source repository with tests and lockfiles, the fictional CSV/XLSX samples and edge cases, Docker files and the documentation in docs/.

Source map

Area	Location
UI	app/static/
API and access	app/main.py
Parsing and DST	app/parsing.py
Rules and engine	app/ruleset.py, app/engine.py
Timeline	app/timeline.py
Storage	app/repository.py, app/schema.py, app/migrations/, app/filestore.py
Tests	tests/
Samples	samples/
This handbook	scripts/build_handbook.py, scripts/capture_handbook_assets.mjs

Maintaining this document

Update this handbook when the API, rules, storage, access or deployment change. Refresh the images with npm run handbook:assets against an empty database, rebuild with uv run --group docs python scripts/build_handbook.py, and review the rendered pages.